ANANTA HACKATHON 2026 · SANGAM · ROUND 2

ECOGUARD AI

UI/UX Design Pack

Energy and water waste detector for an academic block: classrooms, a computer lab and offices.

Team	Team 11, Viltrumites
Problem statement	PS 11: Intelligent Energy and Water Waste Detector for Buildings
Contents	Design goals and rationale · Personas · Sitemap · User flows · DFD Level 0 and 1 · Wireframes (10 screens) · High-fidelity screens · Style guide · States and accessibility · Tool workflow
Live prototype	React dashboard connected to the simulated building (see README in the ecoguard folder of our repository)

1. Design goals and rationale

Our Round 1 plan covered every kind of building and every kind of waste. The jury's main feedback was that it was too broad. For Round 2 we fixed one setting that everyone can picture, **one academic block with five classrooms, a computer lab and two offices**, and designed for every connected energy and water waste inside it.

Decision	Why
Building map is the first thing on the home screen	A facility manager thinks in rooms and floors, not in meter IDs. Colour shows what is happening in each room (in use, empty, wasting, wiring risk), so the problem is found in one look.
Every alert says what, where, since when, what it costs in rupees, and what to do	Studies of feedback systems show that people act on money and clear next steps, not on kWh charts. The rupee figure is computed in code, never guessed by a language model.
Advice in English and Hindi, one tap to switch	Facility and maintenance staff on Indian campuses are often more comfortable in Hindi.
Two buttons on every alert: "Correct" and "Not waste"	Each answer becomes a label that retrains the model for this building. "Not waste" also stops any automatic action, so people stay in control.
Automatic mode with a 15-minute grace period	Fast enough to stop most waste, slow enough for a manager to stop a wrong action. It never switches off a room with people in it.
A separate "Heat in Walls" screen	This is our novel module. Waste that no normal meter shows (hot wiring joints, current leaking into damp walls) needs its own explanation, with a picture of how it is measured.
Energy and water on the same screens, with the pump in the middle	The pump is where energy and water meet. An overflowing tank wastes both, so we show them together instead of in two separate apps.
Savings page with a full-year before vs after	Answers the estate officer's real question: did it work, and how much did it save?

Round 1 plan → Round 2 screens (consistency check)

Module in our Round 1 PPT	Where it lives now
Data collection: IoT sensors, meters, CCTV	Simulator (virtual ESP32 + occupancy) · Rooms · Water & Pump
Intelligence engine: anomaly detection	Alerts · Overview map · Room detail chart (actual vs expected)
Intelligence engine: forecasting	Forecast
Intelligence engine: cross-utility (energy + water)	Water & Pump (pump links both) · Heat in Walls (damp wall + leak)
Decision & control automation (HVAC, lighting, water)	Alert actions · Settings (auto mode, grace period, tickets)
User interface: portal, reports, AI insights	All screens · advice in English/Hindi · labelled CSV export
Savings & carbon (ER entity)	Savings & Carbon (live + full-year before vs after)
Fail-safe: manual override, safe rules	Settings toggles · 'Not waste' stops auto action · relays never cut occupied rooms

2. Personas

Proto-personas built from our literature review of campus energy and water studies and the roles that run our own university blocks.

Persona

Facility Manager

Runs the block day to day; looks at the phone between rounds.

Goals

- Know which room is wasting right now, without walking every floor
- Act in one tap, and not be blamed for switching off a class

Pain points

- Finds out about leaks from the electricity/water bill a month later
- Guard rounds catch lights at 10 pm, too late

Served by: Overview, Alerts, one-tap Switch off, Hindi advice

Persona

Estate Officer / Admin

Owns the budget and the green-campus report.

Goals

- Show money and CO_2 saved with numbers that hold up
- Decide where to spend on repairs

Pain points

- No sub-metering: one bill for the whole block
- Cannot prove that an intervention worked

Served by: Savings & Carbon (before vs after), Forecast

Persona

Electrician / Plumber

Gets called after something breaks.

Goals

- Get told before it breaks, with the exact room or floor
- Fewer emergency calls

Pain points

- Loose joints are found only after a burn smell or a trip
- Leaks inside walls are found only when paint peels

Served by: Heat in Walls, work-order tickets

Persona

Faculty / Students (occupants)

Use the rooms; not dashboard users.

Goals

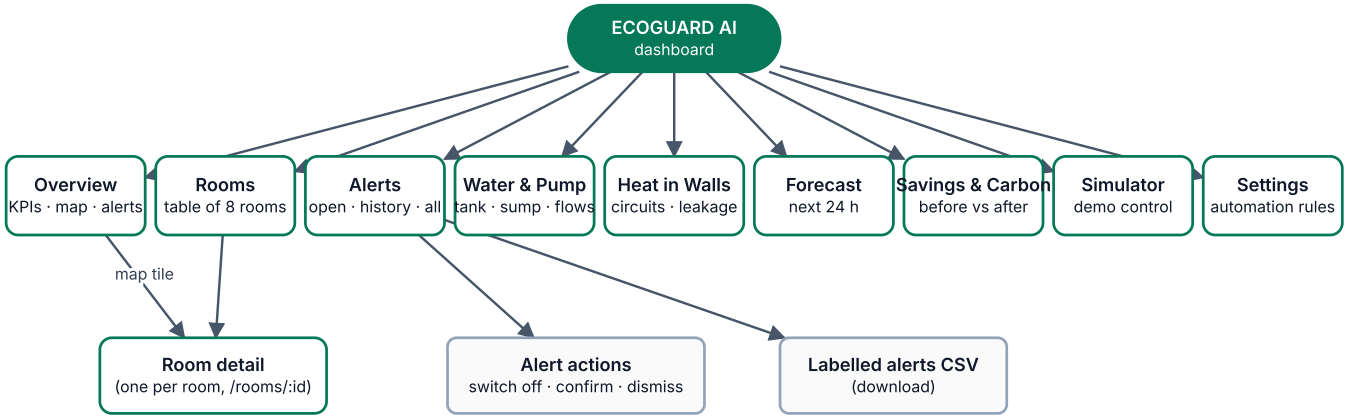
- Room is ready and comfortable when class starts
- Never lose power mid-class

Pain points

- AC set to 16 °C, doors open
- Lights left on by the previous class

Served by: Nothing to learn: ECOGUARD never switches off an occupied room

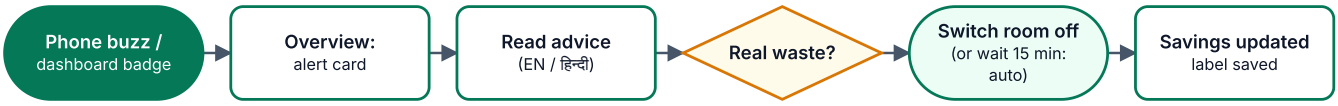
3. Sitemap



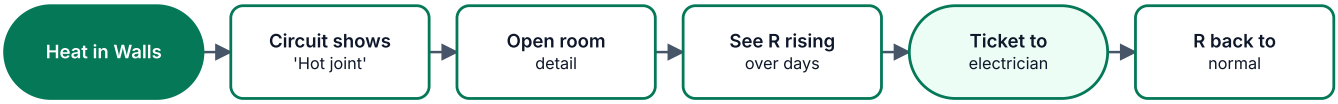
Nine top-level screens in a fixed sidebar (a drawer on phones). Room detail opens from the Rooms table or from any tile on the building map. Every screen is at most two clicks from Overview.

4. User flows

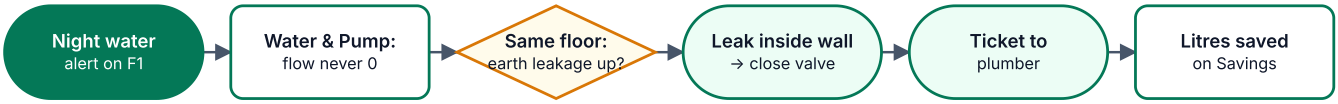
Flow A: respond to waste (facility manager), 2 taps



Flow B: catch a hot wiring joint before it fails (electrician)



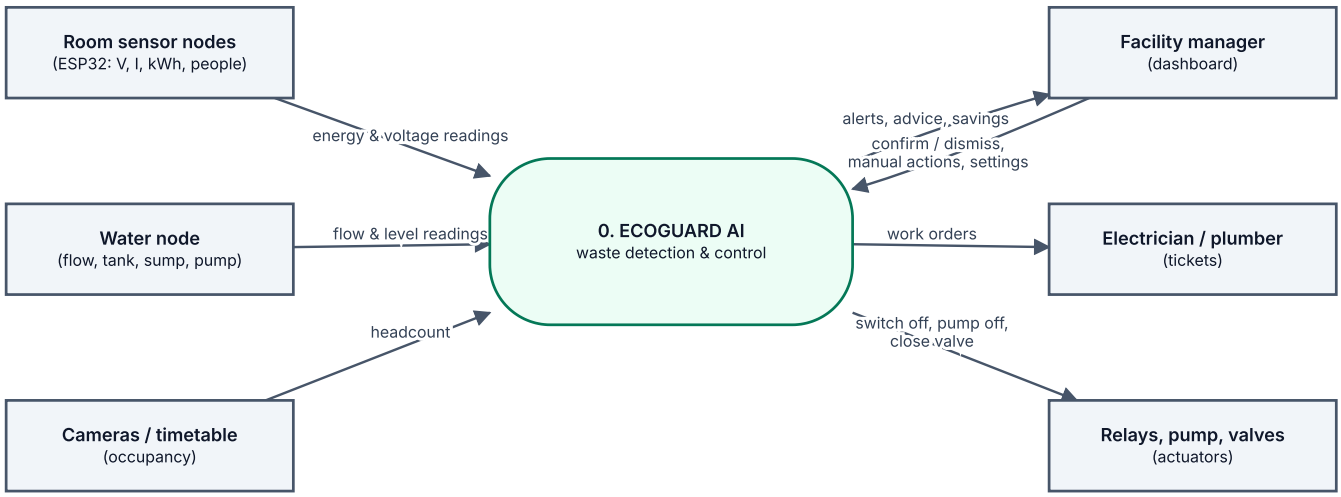
Flow C: find a leak hidden inside a wall (plumber)



Flow D: report savings (estate officer)

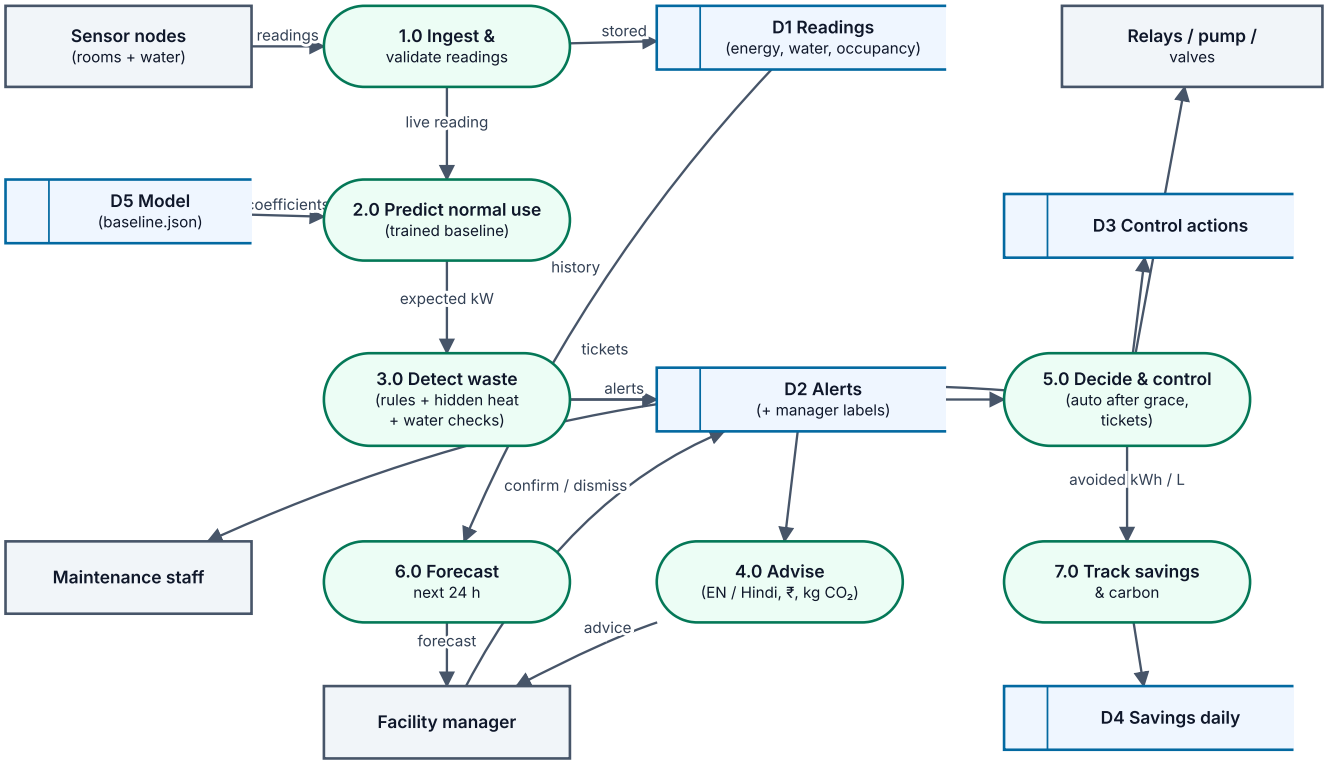


5. Data flow diagram: Level 0 (context)



The whole system as one process. Three kinds of sensors feed it; it talks to people (the facility manager and maintenance staff) and to actuators (relays, pump, valves).

6. Data flow diagram: Level 1

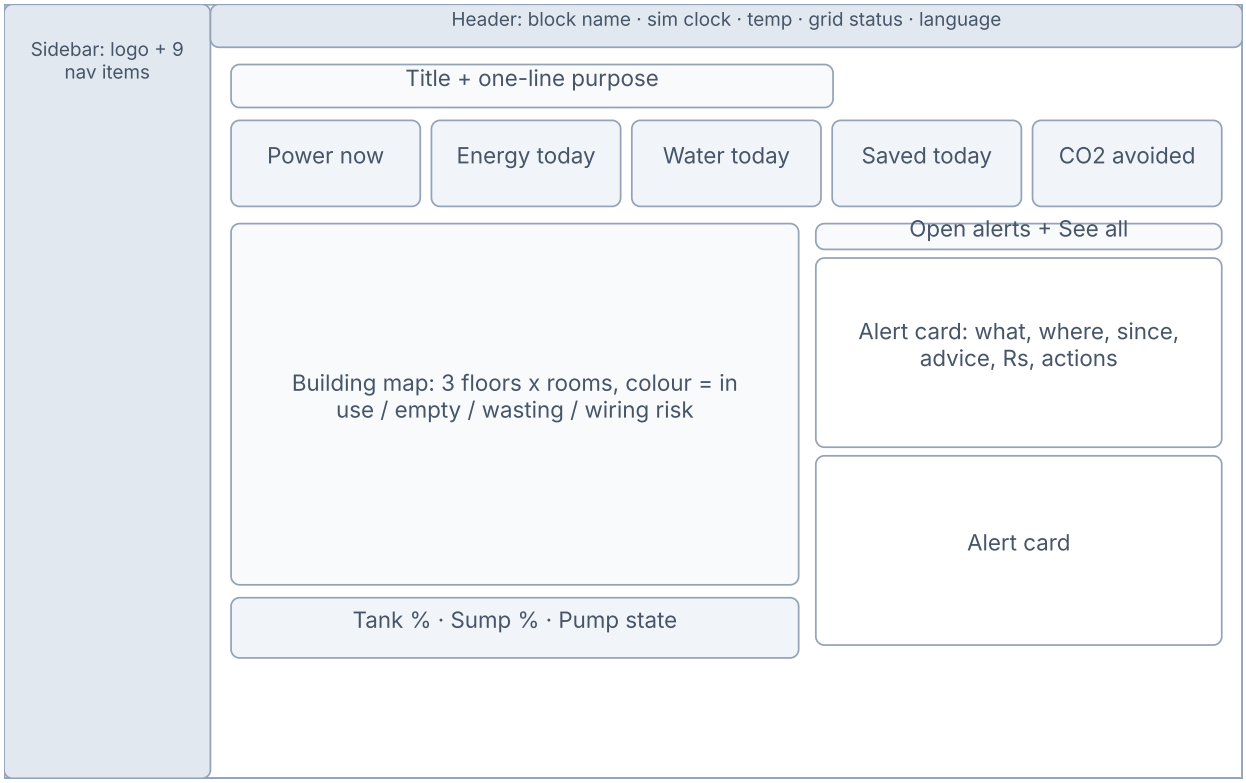


D1 to D4 are tables in our database (energy_readings, water_readings, occupancy_records → D1; alerts → D2; control_actions → D3; savings_daily → D4). D5 is the trained model file.

7. Wireframes (low fidelity)

Desktop frame 1440 × 900. The grey sidebar and header are shared by every screen. Dashed boxes are charts.

W1. Overview



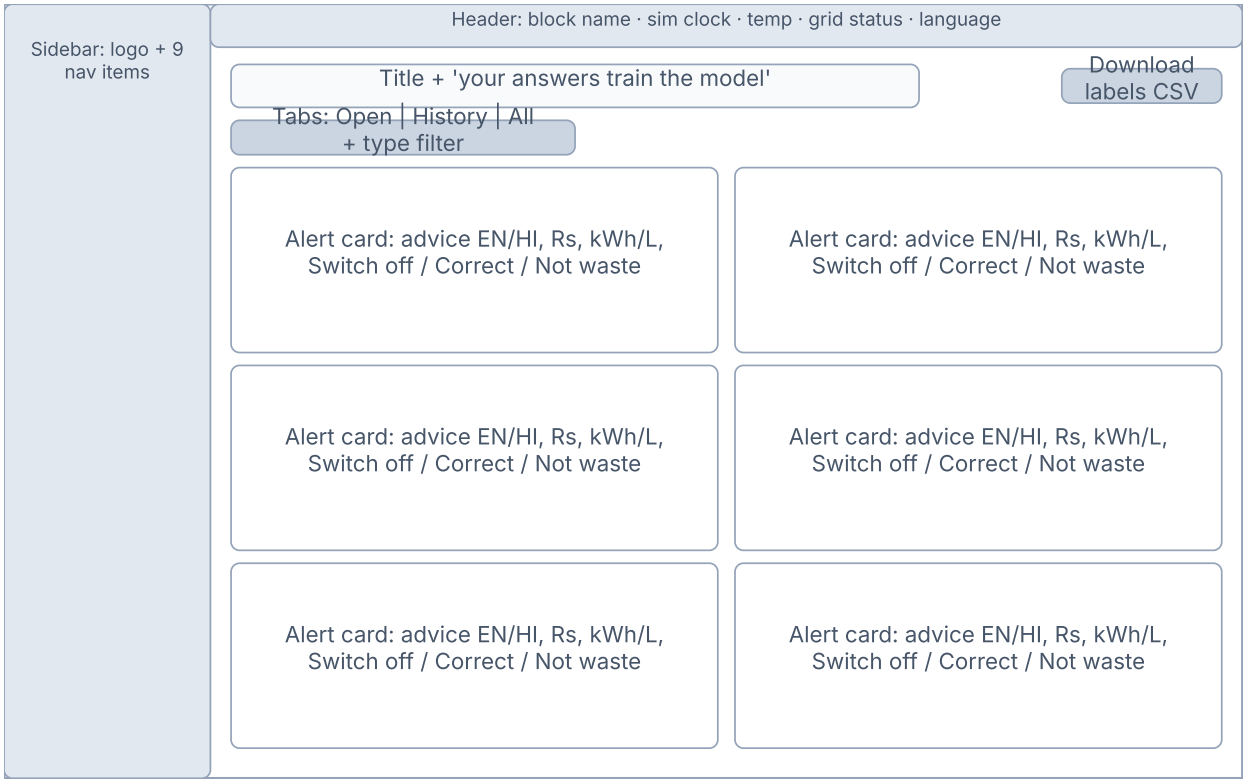
W2. Rooms



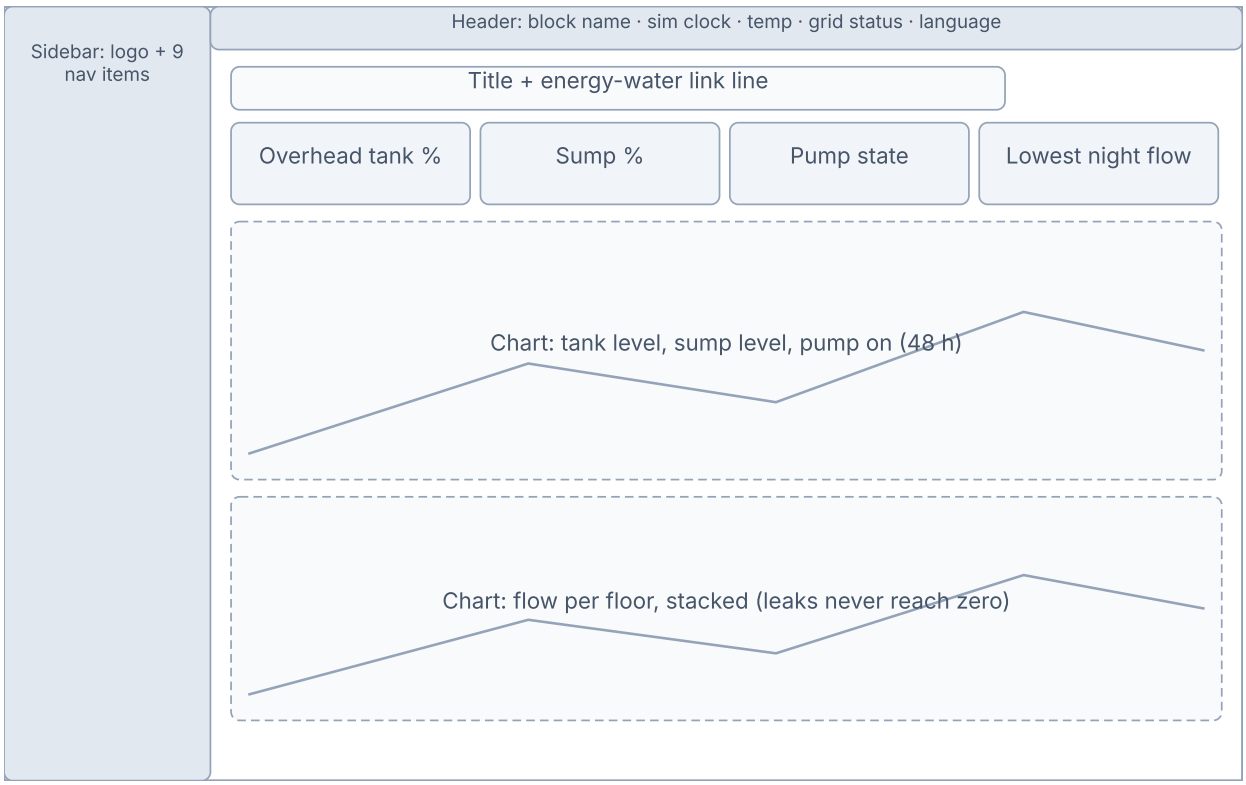
W3. Room detail



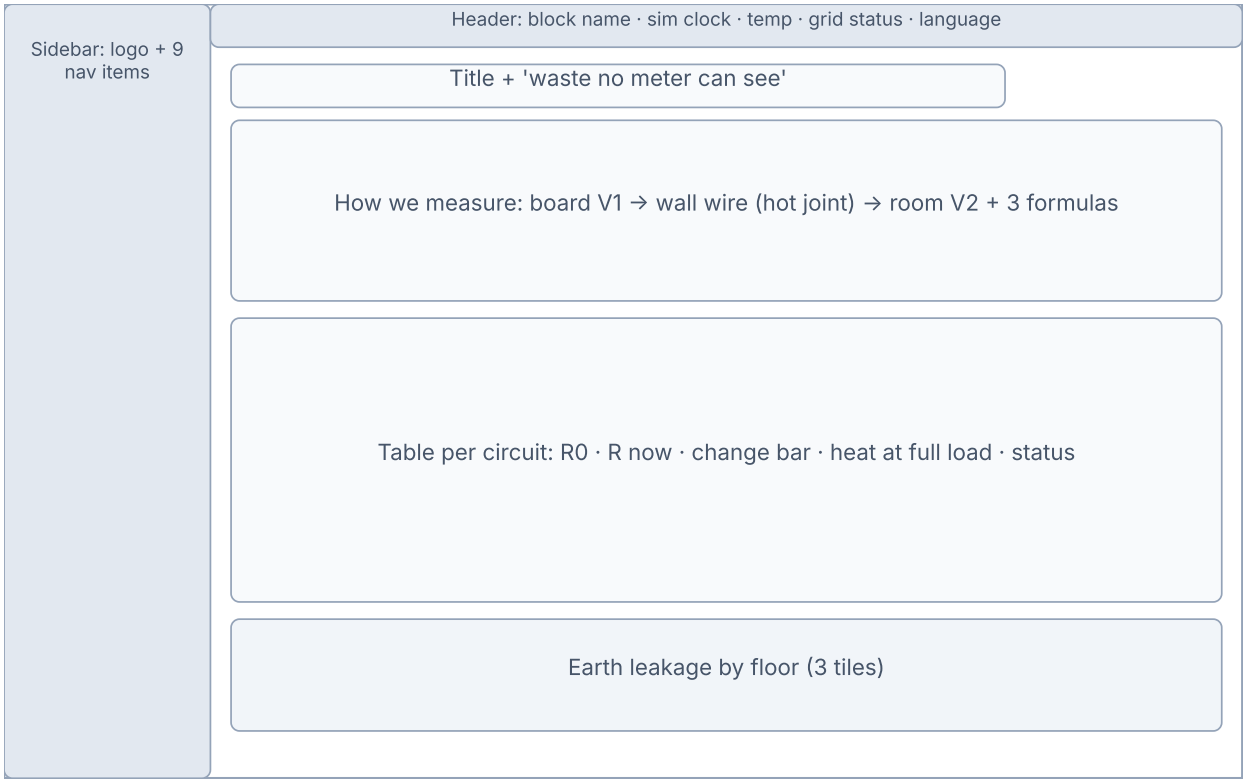
W4. Alerts



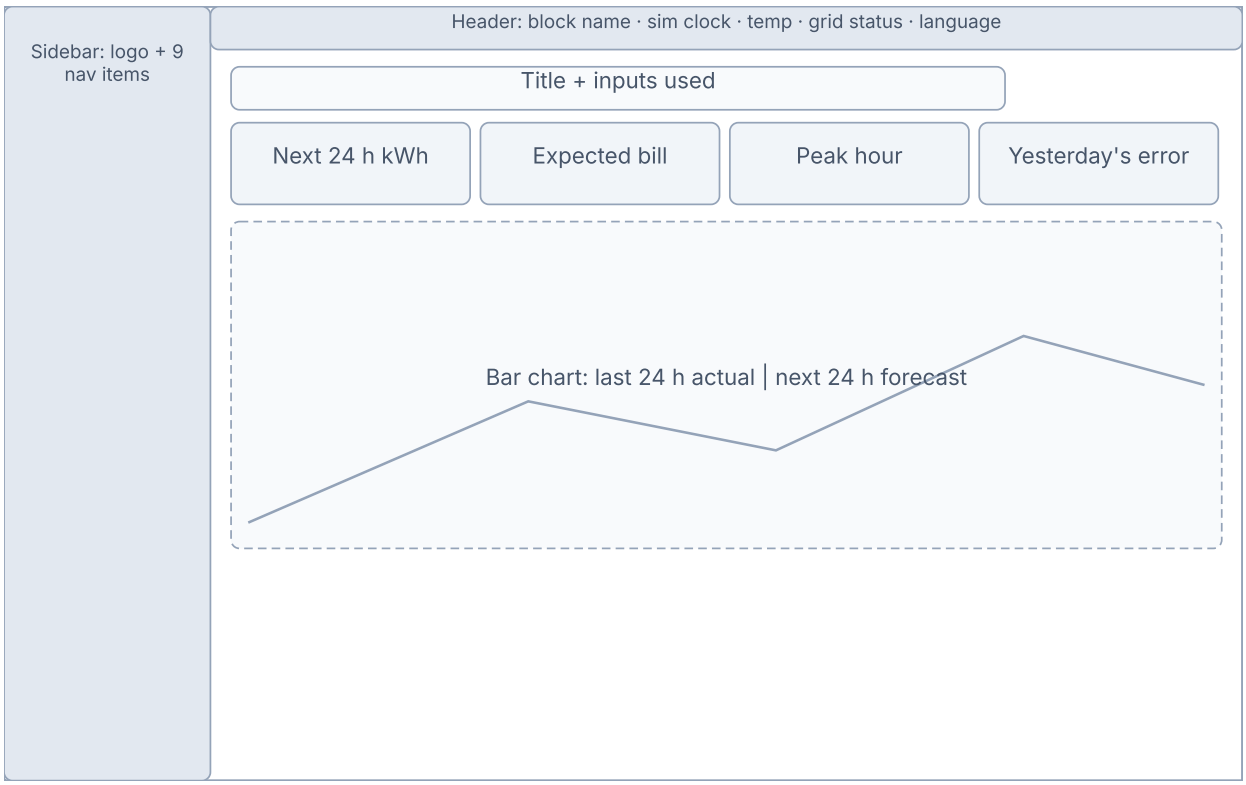
W5. Water & Pump



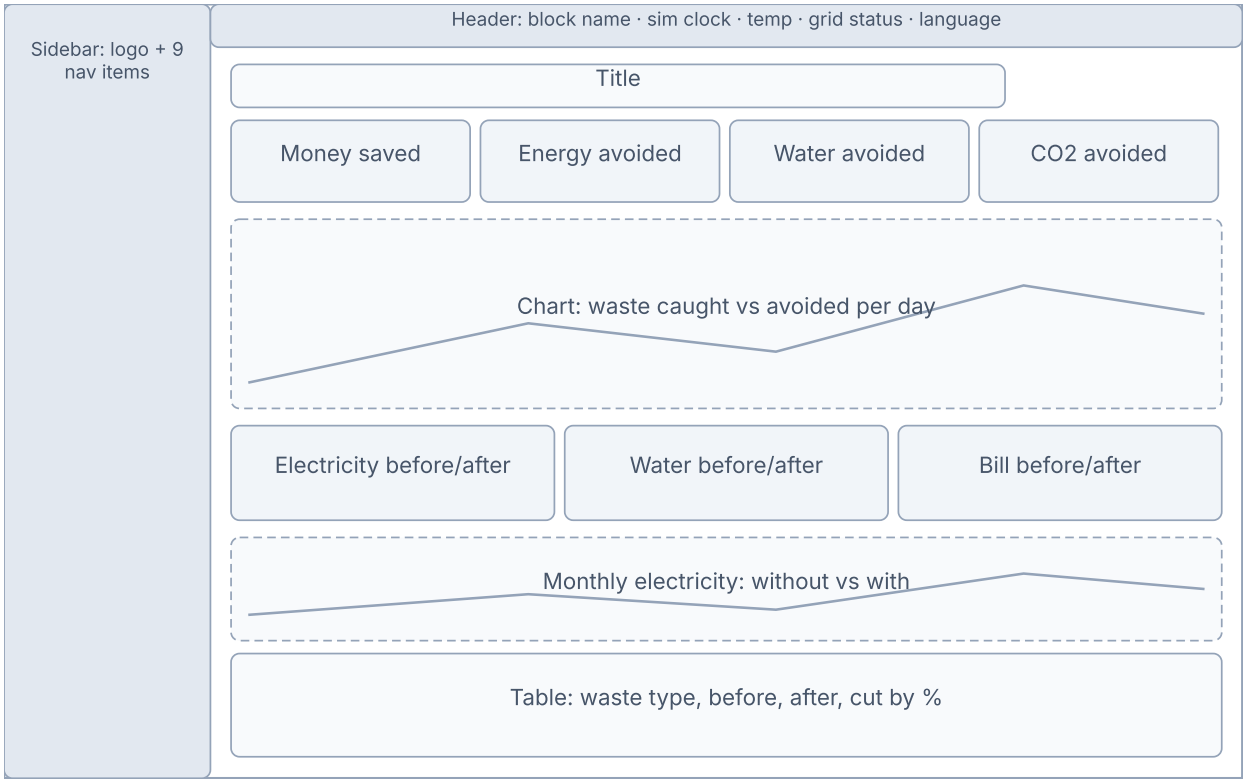
W6. Heat in Walls



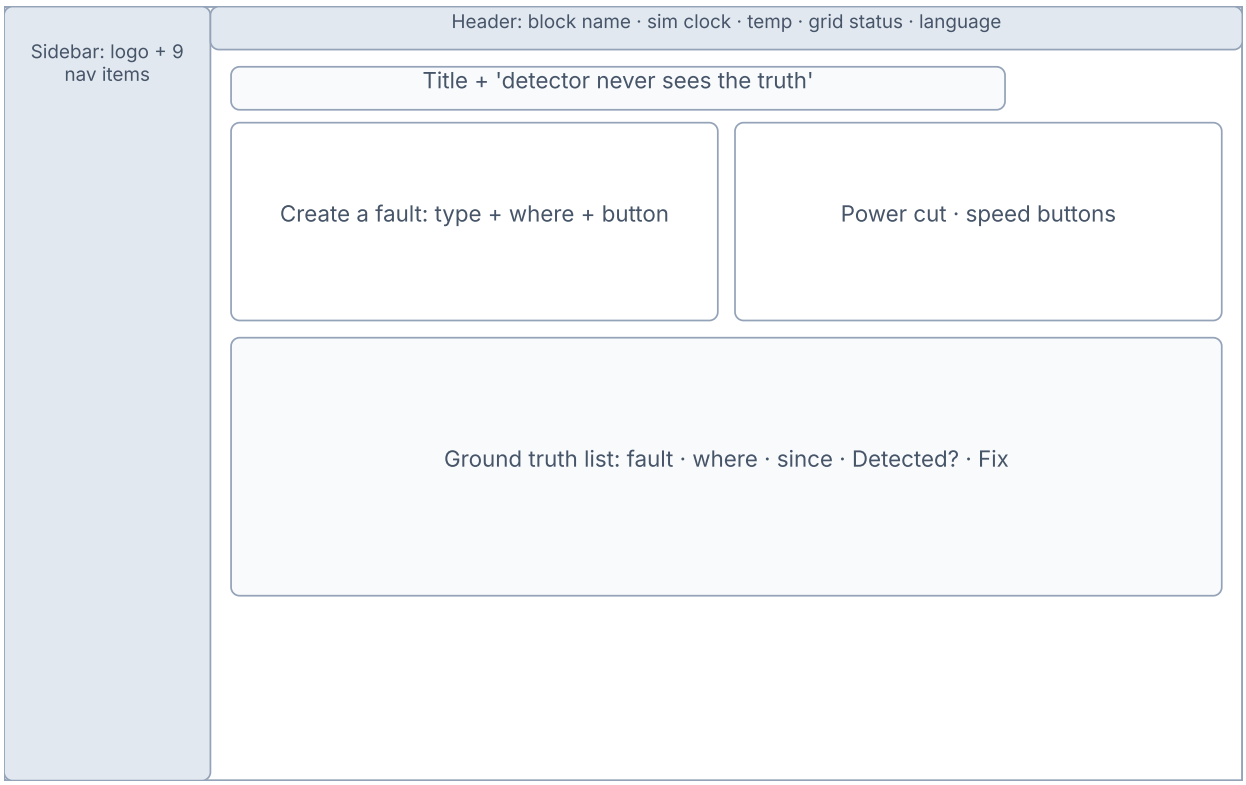
W7. Forecast



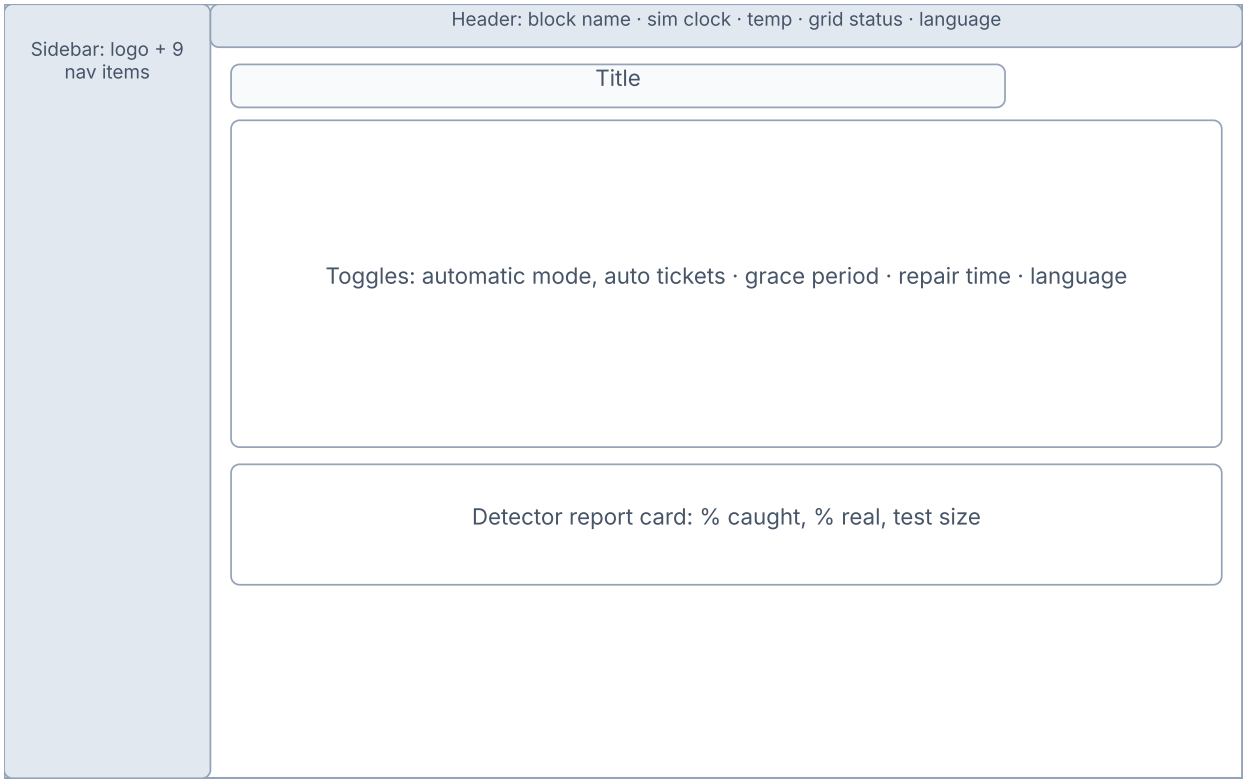
W8. Savings & Carbon



W9. Simulator



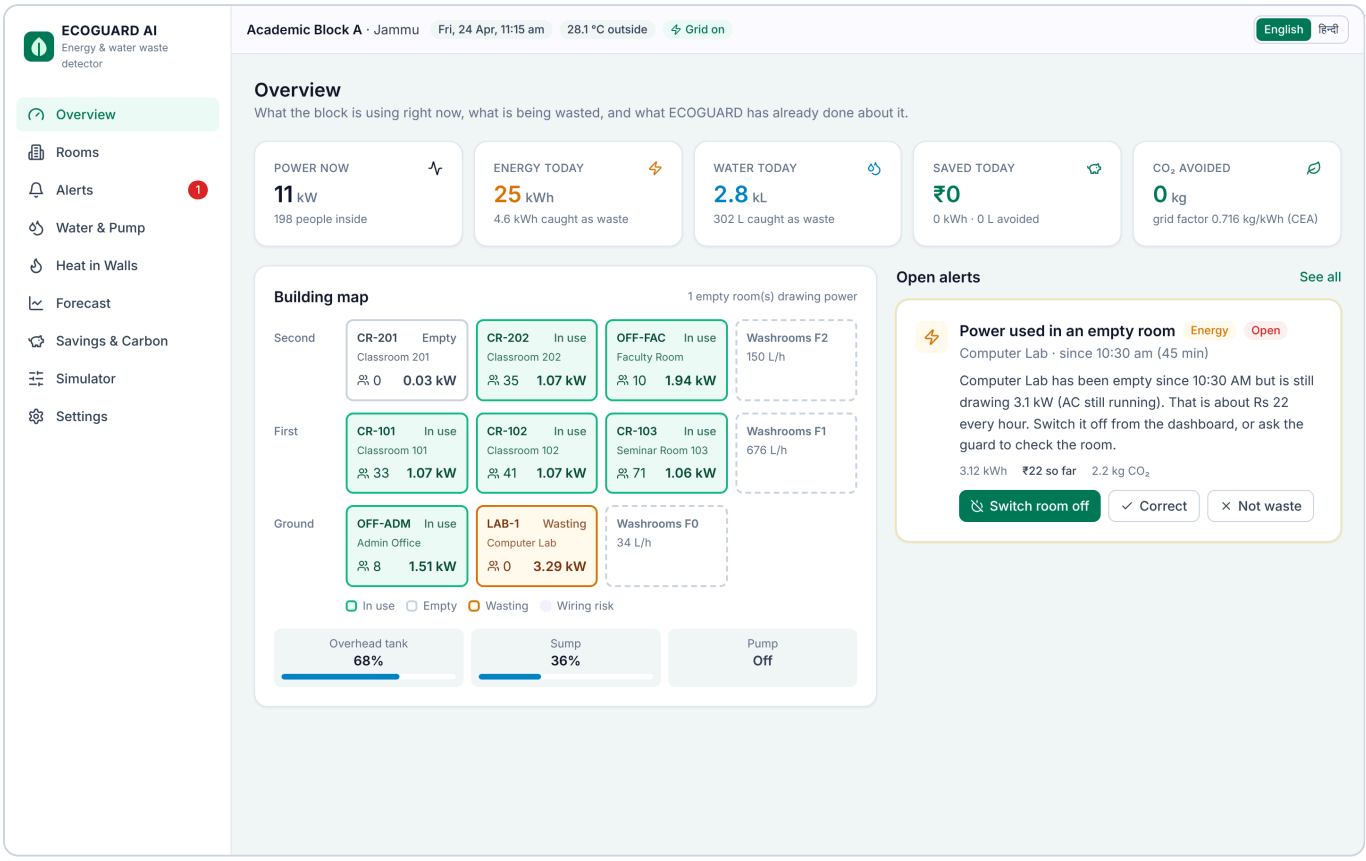
W10. Settings



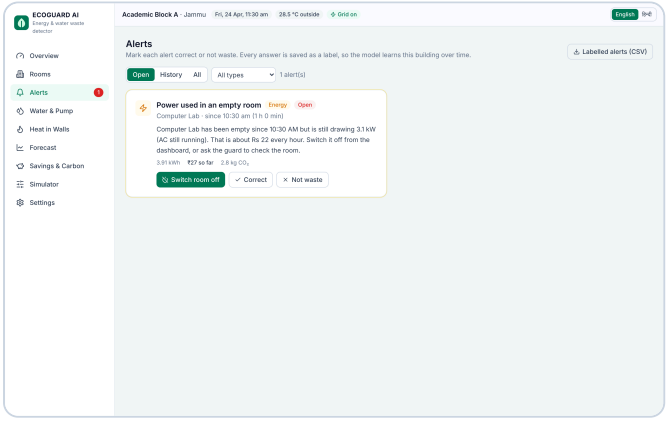
8. High-fidelity screens (working prototype)

Screens captured from the running prototype, fed by the simulated building.

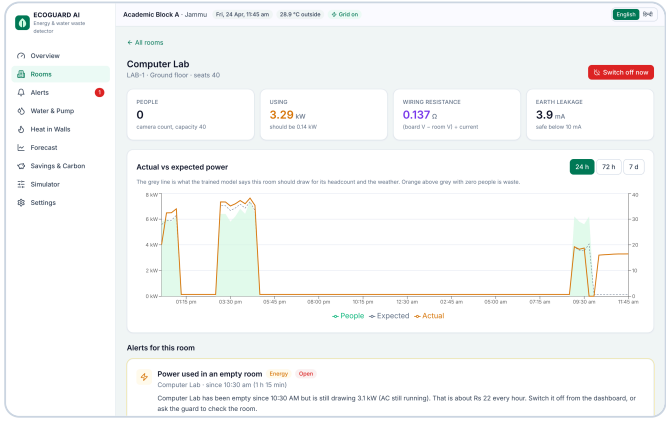
Overview: building map, KPIs and the open alert



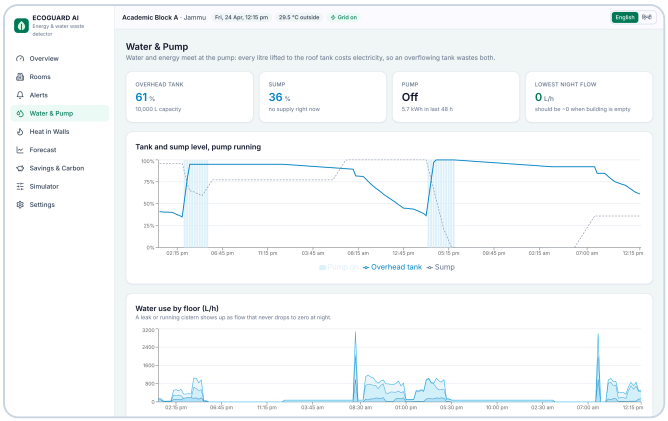
Alerts: advice, cost and feedback buttons



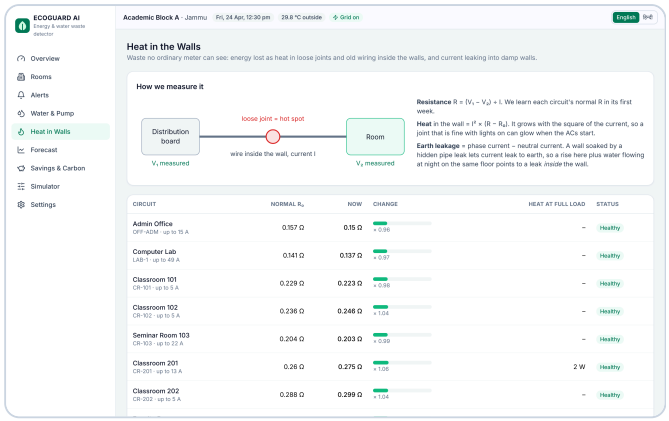
Room detail: actual vs expected use



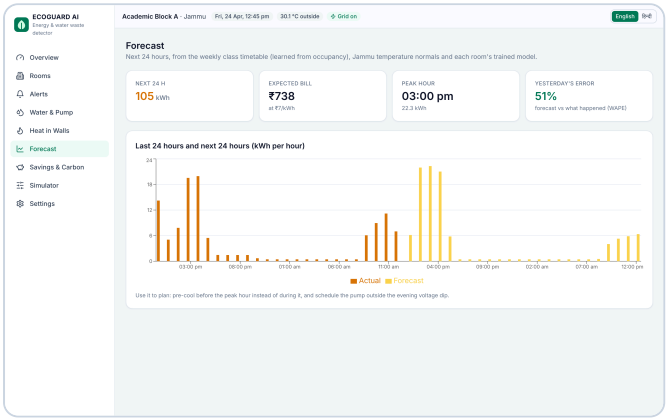
Water & Pump: tank, sump and night flow



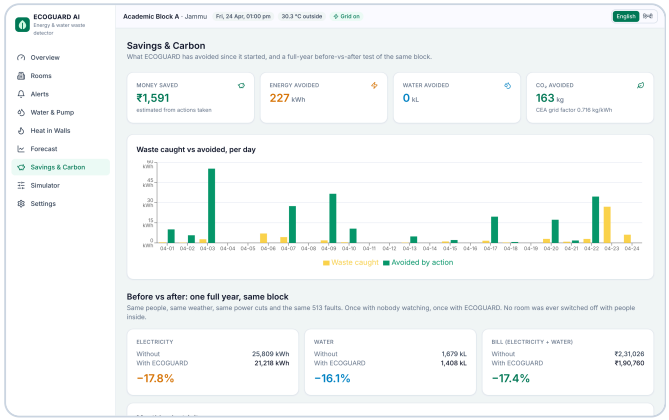
Heat in Walls: wiring resistance and earth leakage



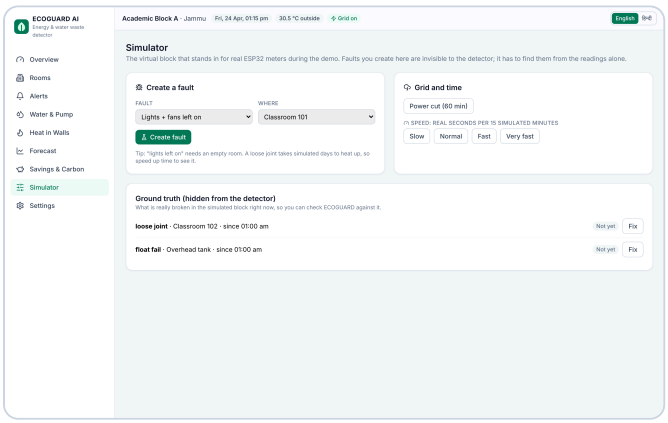
Forecast: next 24 hours



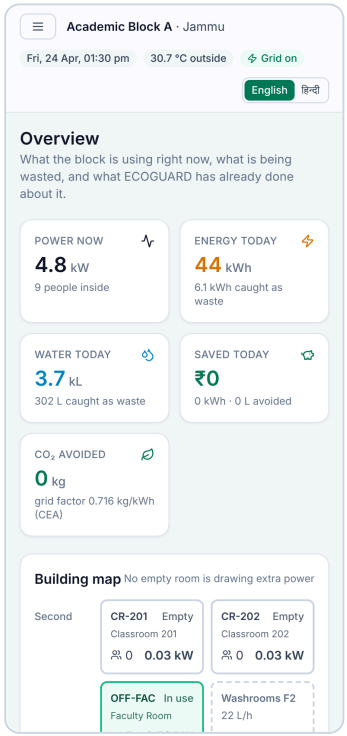
Savings & Carbon: full-year before vs after



Simulator: demo controls



Phone view



9. Style guide

Colour

brand-700 #047857
Primary actions, active nav, positive numbers

brand-50 #ecfdf5
Selected nav, 'in use' rooms

energy #d97706
Electricity values, energy alerts

water #0284c7
Water values, water alerts

hidden #7c3aed
Wiring heat / earth leakage (hidden waste)

danger #dc2626
Open alerts, switch-off, power cut

ink-900 #0f172a
Headings, key numbers

ink-500 #64748b
Secondary text

ink-100 #f1f5f9
Page background, tiles

Each resource has one colour everywhere: energy amber, water blue, hidden waste violet. Text contrast is at least 4.5:1 on white.

Typography

Inter 20 semibold: page title

Inter 16 semibold: card title

Inter 14 regular: body and advice text

INTER 12 MEDIUM: LABELS

Spacing and shape

- 4 px grid; card padding 16-20 px; gap 12 px; page padding 24 px
- Card radius 14 px, button radius 8 px, pills fully round
- One shadow level (cards only)

Icons

Lucide line icons at 16-18 px: Zap (energy), Droplets (water), Flame (wall heat), Bell (alerts), PowerOff (switch off), Wrench (ticket), Leaf (carbon).

Components

- **KPI card**: label, big number + unit, one-line context
- **Alert card**: kind icon, title, kind pill, status pill, place and time, advice, cost line, actions
- **Room tile**: 4 states (in use, empty, wasting, wiring risk)
- **Buttons**: primary, ghost, danger; disabled when the action makes no sense (e.g. switching off an occupied room)
- **Toggle** (role=switch), **tabs**, **select**, **meter bar**

10. States and accessibility

State	What the user sees
Loading	Spinner with "Loading the building…" (announced to screen readers)
Empty	"Nothing is being wasted right now", "No readings yet: start the gateway", "Before-vs-after not run yet"
Error	Red banner "Could not reach the ECOGUARD server" with a Retry button; header pill "Backend offline"; live view falls back to polling every 5 s

Power cut	Header pill turns red "Power cut"; energy rules pause (meters read zero), water rules continue

- Colour is never the only signal: every room tile and alert also has a text label.
- All charts have text KPIs next to them; meters use role="meter" with values.
- Keyboard: every control is a real button or link with a visible focus ring.
- Readable sizes: body 14 px, minimum 11 px for chart ticks; layout works down to 390 px phones.

11. Tool workflow

We built the clickable prototype directly in code (React + Tailwind + Recharts), connected to live simulated data, so every screen shows realistic numbers and every button really works. The design tokens live in `tailwind.config.js` and are the same names as in this guide. To present the design file, the screens are rebuilt in Figma as follows:

1. Make one page per screen in the sitemap, each a 1440 × 900 desktop frame, plus one 390 × 844 phone frame for Overview.
2. Create colour styles from the table above (brand, energy, water, hidden, danger, ink) and text styles: H1 20/28 semibold, H2 16 semibold, body 14, small 12, label 12 uppercase.
3. Build components with auto-layout: Sidebar item (variants: default / active / with badge), KPI card, Alert card (variants: energy / water / hidden × open / actioned / dismissed), Room tile (in use / empty / wasting / wiring risk), Button (primary / ghost / danger), Pill.
4. Use a 4 px spacing grid: 16 px card padding, 12 px gaps between cards, 24 px page padding.
5. Assemble screens from components only (no loose rectangles), so a jury member can click any element and see it is an instance.
6. Prototype links: sidebar items → screens; room tile → Room detail; alert 'Switch room off' → overlay toast 'Switch-off sent'; 'Not waste' → card variant Dismissed. Use Smart Animate, 200 ms ease-out.
7. Design the three states on Overview and Alerts: Loading (skeleton), Empty ('Nothing is being wasted right now'), Error ('Could not reach the ECOGUARD server' + Retry).
8. Hand-off: turn on Dev Mode; tokens map 1:1 to tailwind.config.js names in the code.